

Manual técnico — API, REVERSE-PROXY y CLIENTE DE REACT

Financiamiento de Cuentas por Pagar. Este documento explica cómo se instalan y cómo se conectan los tres proyectos.

Pieza	Carpeta	Stack
API	CXP_DAV_SERVER_REFACTOR	Java 21, Spring Boot 3.3.0, PostgreSQL 16. Empaquetado <code>war</code> . Artefacto <code>APIFinanciamientoEmpresas</code> . El starter de Tomcat va en alcance <code>provided</code> : en el ambiente corre en un Tomcat externo.
Portal	financiamientocuentasporpagar	Java 21, Spring Boot 3.4.4, Tomcat 11.0.4 (<code>provided</code>). Empaquetado <code>war</code> , nombre final <code>financiamientocuentasporpagar</code> . Recibe la sesión del portal del banco, pide el JWT a la API y sirve el frontend.
Frontend	CXP_DAV_CLIENT_REFACTOR	React 19, Vite 6, Tailwind 4. Node.js 18 o superior. Se compila y el resultado se copia dentro del WAR del portal.

La API publicada que usa el cliente está en `https://dev-financiamientocuentasporpagar-api.davivienda.com.sv:8443/APIFinanciamientoEmpresas`. El frontend se publica bajo `/financiamientocuentasporpagar/`.

1. Base de datos

Los dos WAR usan el mismo datasource JNDI: `java:comp/env/jdbc/FinanciamientoCuentasDB`.

El DBA ejecuta una sola vez `CXP_DAV_SERVER_REFACTOR/db/prod/instalacion_inicial.sql` sobre una base PostgreSQL 16 vacía. El script deja los catálogos, la entidad banco y los dos usuarios madre.

Tabla de sesión SSO

El handoff guarda el token de un solo uso en `sso_pending_sessions` (`PendingSessionModel`): `session_token` (clave), `user_email`, `created_at` y `expires_at`.

Parámetros de SSO

Estas filas no las inserta el script. Van en `system_parameters`. La pantalla de parámetros puede editarlas si la fila ya existe; no las da de alta.

Clave	Quién la lee	Uso
BEARER_TOKEN	API, en /api/v1/sso/handoff	Debe coincidir con el encabezado Authorization: Bearer <valor>
APP_CODE	API, en el handoff	Debe coincidir con el campo app del cuerpo
PAY_BEARER_TOKEN	API	Se envía a Davivienda como X-Authorization: Bearer <valor>
PAY_DAVIVIENDA_URL	API	URL a la que la API postea { "otc": "..."} para validar el OTC
FIN_CXP_URI	WAR del portal	URL absoluta de POST /api/v1/sso/activateSession de la API

2. Compilación de API

Empaquetar desde CXP_DAV_SERVER_REFACTOR:

```
mvn clean
```

```
mvn install
```

El WAR queda en target/APIFinanciamientoEmpresas-0.0.1-SNAPSHOT.war. ServletInitializer y debe renombrarse a APIFinanciamientoEmpresas.war, posteriormente se debe desplegar en el Tomcat externo correspondiente.

3. Compilación del cliente de REACT (Frontend)

Las direcciones están fijadas en CXP_DAV_CLIENT_REFACTOR/src/constants/apiConstants.js.

Constante	Valor actual	Uso
API_BASE_URL	https://dev-financiamientocuentasporpagar-api.davivienda.com.sv:8443/APIFinanciamientoEmpresas	Origen de la API, con context path y sin /api
API_URL	{API_BASE_URL}/api	Base de axios
PORTAL_URL	https://devpay.davivienda.com.sv	A dónde va el navegador sin sesión o al salir

Compilar desde CXP_DAV_CLIENT_REFACTOR:

```
npm install
npm run build
```

El contenido de la carpeta `dist/` se debe copiar dentro de

`financiamientocuentasporpagar/src/main/resources/static/` (el `index.html` en la raíz de `static/` y la carpeta `assets/`). Esos estáticos ya traen una compilación previa; hay que reemplazarlos cuando cambie el cliente.

El origen del front publicado (el host del WAR) también tiene que estar en `CORS_ALLOWED_ORIGINS`. Se cambia en Gestión de parámetros (rol `SYSTEM_ADMIN`) y aplica sin reiniciar la API.

4. Compilación del REVERSE-PROXY

Empaquetar desde `financiamientocuentasporpagar`, después de copiar el `dist/` del cliente:

```
mvn clean
```

```
mvn install
```

El WAR se almacena en `target/financiamientocuentasporpagar.war`. Usa el mismo JNDI

`java:comp/env/jdbc/FinanciamientoCuentasDB` para leer `FIN_CXP_URI` de `system_parameters`.

Un 404 de una ruta del cliente se reenvía a `index.html` con estado 200, para que el router de React resuelva la pantalla.

El ingreso es `POST /start-session` con el parámetro `sessionToken`. El controlador lee `FIN_CXP_URI`, postea `{ "sessionToken": "...", "success": true }` y espera `{ "success", "message", "jwt" }`. La respuesta al navegador es un HTML que guarda el JWT y redirige:

```
localStorage.setItem('jwt_token', '<jwt>');  
window.location.href = '/financiamientocuentasporpagar/';
```